IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 25 July 2025, accepted 15 August 2025, date of publication 19 August 2025, date of current version 28 August 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3600283

RESEARCH ARTICLE

DC-RRT*: Dubins-Guided Curvature RRT* for 3D Path Planning of Unmanned Aerial Vehicles

YEKYUNG JUNG[1] AND BEOM-SEOK OH[2], (Member, IEEE)
[1]Department of Defence AI Convergence Engineering, Seoul National University of Science and Technology, Seoul 01811, Republic of Korea
[2]Department of Applied Artificial Intelligence, Seoul National University of Science and Technology, Seoul 01811, Republic of Korea

Corresponding author: Beom-Seok Oh (bsoh@seoultech.ac.kr)

This work was supported by the National Research Foundation of Korea (NRF) grant funded by Korean Government (MSIT) under Grant RS-2023-00221365.

ABSTRACT In this paper, we propose a Dubins-guided curvature rapidly exploring random tree* (DC-RRT*), a novel path planning method for unmanned aerial vehicles (UAVs). Different from conventional sampling-based path planning methods, DC-RRT* explicitly considers UAV-specific physical constraints, such as turning radius and vehicle size. Particularly, it employs a curvature-based node expansion strategy that explicitly accounts for both the UAV's heading direction and minimum turning radius. The method enhances path convergence by attempting direct Dubins path connections between newly generated nodes and the target, while ensuring feasibility through comprehensive full-body collision checking. Our simulation study conducted on four obstacle scenarios, demonstrates that the proposed DC-RRT* generates paths with significantly fewer nodes than existing benchmarking methods, while maintaining computational efficiency.

INDEX TERMS Unmanned aerial vehicle, path planning, Dubins path, rapidly exploring random tree.

I. INTRODUCTION

Unmanned aerial vehicles (UAVs) possess high mobility and sensor-based data acquisition capabilities, enabling them to operate effectively in environments that are challenging for human access [1], [2]. Leveraging these advantages, UAVs have been successfully applied across a broad spectrum of missions, ranging from large-scale operations such as disaster monitoring [3], maritime patrol [4], agricultural management [5], and border security [6], to precision tasks in narrow and complex environments [7].

Despite their versatility, fixed-wing UAVs inherently lack the capability to hover or make sudden directional changes. Even rotary-wing UAVs, which exhibit superior maneuverability, face limitations in performing abrupt turns due to inertia and restricted actuation capabilities. Consequently, establishing effective flight paths in complex terrains such as mountainous regions or densely built urban areas, remains challenging [8]. Therefore, path planning methods must explicitly account for dynamic and kinematic constraints, including minimum turning radius and orientation continuity, ensuring generated paths are physically feasible by UAVs.

According to [9], existing UAV path planning methods can be categorized into three approaches: grid search-based, model-based, and sampling-based. Methods belonging to the grid search-based approach, such as Dijkstra and A*, are conceptually intuitive and thus straightforward to implement [10]. However, these methods incur high computational costs, which rise sharply as the search space expands [11]. In contrast, model-based methods accurately derive optimal paths through mathematical formulations, but their real-time applicability in dynamic environments is limited due to significant modeling and computation time [12]. Sampling-based methods, such as rapidly exploring random tree (RRT) [13], offer distinct advantages over both approaches owing to their lower computational cost, making them particularly effective for efficiently exploring paths in complex environments.

Although RRT can generate paths in real-time, it could not guarantee that the resulted paths are optimal [14]. RRT*

The associate editor coordinating the review of this manuscript and approving it for publication was Guillermo Valencia-Palomo.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

[15], which improves upon RRT, converges asymptotically to optimal paths through tree rewiring and cost re-evaluation. However, as the number of nodes increases, the computational costs of RRT* escalate significantly, causing delays and thus limiting its applicability in complex and real-time applications [16]. To accelerate convergence, Biased RRT [17] increases the likelihood of expanding nodes toward the target point. Yet, this direct expansion may decrease search efficiency in obstacle-rich environments [18]. Therefore, enhancing sampling-based methods to balance computational efficiency with path quality is crucial.

To address these challenges, various variants of the RRT have recently been proposed [19], [20], [21], [22], [23], [24]. To enhance computational efficiency, Ichter and Pavone [19] proposed reducing the geometric search space by employing a learned latent space representation. Adapted-RRT [21] integrates the grey wolf optimization metaheuristic algorithm into RRT to minimize unnecessary random exploration. Learning-based RRTs [20], [22], [23] have also been explored, utilizing neural networks to generate paths. More recently, an adaptive bi-directional RRT algorithm has been introduced, employing an adaptive sampling strategy and adjustable step size to accelerate convergence in UAV path planning in 3D environments [24].

Potential field-based RRT* (PF-RRT*) [18] and artificial PF-RRT* (APF-RRT*) [25] have been proposed by integrating potential field models to enhance the directionality of RRT expansion and improve obstacle avoidance. Although effective in sparse environments, their efficiency significantly declines in dense, cluttered scenarios due to conflicting attractive and repulsive forces. Moreover, typical approaches relying on simple geometric node connections struggle to accommodate UAV-specific constraints, such as the minimum turning radius. The circular arc fillet RRT* (CAF-RRT*) [26] method improves path smoothness through geometric post-processing by replacing sharp transitions with circular arcs. However, because this smoothing occurs after path generation without integrating physical constraints, the resulting paths may still be infeasible for UAVs subject to curvature limitations. Therefore, existing methods inadequately address critical UAV-specific physical constraints.

In this study, we propose a Dubins-guided curvature RRT* (DC-RRT*), a three-dimensional path planning method explicitly incorporating UAV-specific constraints, such as minimum turning radius, heading angle continuity and UAV volumetric dimensions, into tree expansion and node connection processes. Accurate collision detection employs a full 3D volumetric collision-checking model, surpassing simplified bounding shapes or point-based methods. Node connections during tree expansion respect curvature constraints reflecting the UAV's minimum turning radius and heading direction. Additionally, the method evaluates the feasibility of directly constructing Dubins paths with minimum turning radii to the target point, ensuring optimal path planning. Incorporating a Biased RRT strategy by adjusting expansion probabilities around the goal enhances convergence.

The main contributions of this work are summarized as:

- We propose a curvature-based node expansion method that accounts for the physical properties of the UAV, including the airframe dimensions and minimum turning radius, thereby improving path feasibility in 3D space.
- We introduce a Dubins path-based node connection strategy that ensures the generated paths are dynamically feasible and smooth, complying with UAV motion constraints.
- We present a biased expansion technique that attempts to directly connect new nodes to the target using Dubins path, reducing the number of nodes while improving solution quality and accelerating convergence.

The remaining paper is organized as follows In Section II, we provide some background knowledge on RRT methods and Dubins path generation algorithm for immediate reference. Section III presents the details of the proposed DC-RRT* method for 3D path planning. Section IV reports simulation and comparison results under four scenarios with various obstacle configurations. Lastly, concluding remarks are provided in Section V.

II. PRELIMINARIES

In this section, we first provide background knowledge on RRT and its variants, including RRT* and Biased RRT. This is followed by a discussion of the Dubins path model in 2D space, which imposes a minimum turning radius constraint.

A. RRT AND ITS VARIANTS

1) CONVENTIONAL RRT METHOD

RRT is a sampling-based path planning method that incrementally constructs a search tree by randomly sampling the configuration space [13]. The tree is expanded iteratively as follows: at each iteration, a random node $\boldsymbol{q}_{rand}$ is sampled, and the nearest node $\boldsymbol{q}_{nearest}$ in the existing tree is identified. A new node $\boldsymbol{q}_{new}$ is then generated by moving a fixed distance ϵ from $\boldsymbol{q}_{nearest}$ toward $\boldsymbol{q}_{rand}$, and it is added to the tree if the connecting edge does not intersect any obstacles, as illustrated in Fig. 1. This sampling and expansion process is terminated when stopping criteria is satisfied.

2) RRT* METHOD

RRT* is an extension of the RRT method that introduces cost-based procedures namely, ChooseParent and Rewire, to minimize the path length and incrementally converge toward an optimal path. After generating a new node $\boldsymbol{q}_{new}$, RRT* identifies all nearby nodes $\boldsymbol{q}_{near}$ within a predefined radius R_n. The ChooseParent step selects the node from $\boldsymbol{q}_{near}$ that results in the lowest path cost to $\boldsymbol{q}_{new}$, designating it as the parent node of $\boldsymbol{q}_{new}$. Subsequently, the Rewire process evaluates whether connecting each nearby node $\boldsymbol{q}_{near}$ through $\boldsymbol{q}_{new}$ would reduce their existing path costs. If cost improvements are possible, these nodes are reassigned to $\boldsymbol{q}_{new}$. Through iterative application of the ChooseParent and Rewire steps, RRT* progressively approaches an optimal solution.

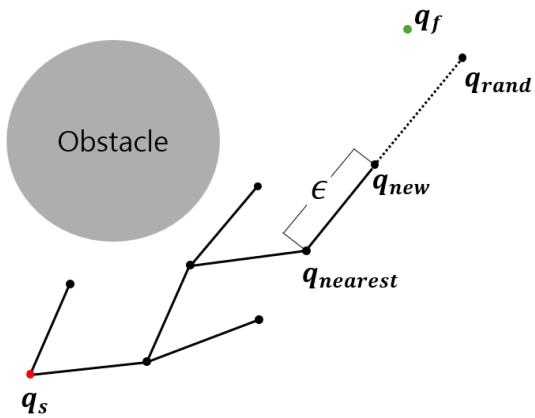


FIGURE 1. An illustration of the conventional RRT process. Symbols q_s and q_f denote the coordinates of starting and target points, respectively.

Although this rewiring process significantly enhances path quality, it increases computational overhead due to neighborhood searches, particularly in high-dimensional spaces.

3) BIASED RRT METHOD

Biased RRT introduces a goal-directed sampling strategy to enhance the convergence rate of RRT. Instead of randomly sampling the state space at each iteration, this method selects the target point $\boldsymbol{q}_f$ as the sample with a probability k. This bias increases the likelihood of tree expansion toward the target point, thus reducing planning time in relatively open environments. However, an excessively high goal bias may hinder exploratory capabilities and result in suboptimal paths, particularly in complex environments, by causing the method to overlook alternative feasible routes. Therefore, the goal bias probability k is typically set between 5% and 10%, depending on obstacle density in the environment [27].

B. DUBINS PATH GENERATION IN A 2D SPACE

The Dubins path model defines the shortest trajectory for a vehicle subject to a minimum turning radius constraint [28], thereby capturing realistic flight characteristics such as the turning behavior of UAVs [29]. Given a starting point and a target point, each specified by both position coordinate ($\boldsymbol{q}_s$ and $\boldsymbol{q}_f$, $\forall \boldsymbol{q} = [x, y]^T$) and heading directions ($\boldsymbol{v}_s$ and $\boldsymbol{v}_f$, $\forall \boldsymbol{v} = [v_x, v_y]^T$), the centers of the left and right turning circles tangent to the direction vector at each point, can be computed. Based on the starting position coordinate and heading vector, these turning centers are defined as follows [30]:

$$\boldsymbol{c}_{\{s,f\},l} = \boldsymbol{q}_{\{s,f\}} + r(\boldsymbol{n} \times \boldsymbol{v}_{\{s,f\}}),$$
$$\boldsymbol{c}_{\{s,f\},r} = \boldsymbol{q}_{\{s,f\}} - r(\boldsymbol{n} \times \boldsymbol{v}_{\{s,f\}}),$$

where $\{\cdot\}$ indicates a selection operation in which $\boldsymbol{c}_{\{s,f\},l}$ indicates either $\boldsymbol{c}_{s,l}$ or $\boldsymbol{c}_{f,l}$, r indicates the minimum turning radius, and $\boldsymbol{n}$ denotes the unit normal vector perpendicular to the plane. The direction vector $\boldsymbol{c}_d$, which connects the selected pair of turning centers, is then defined as follows:

$$\boldsymbol{c}_d = \frac{\boldsymbol{c}_{f,\{l,r\}} - \boldsymbol{c}_{s,\{l,r\}}}{\|\boldsymbol{c}_{f,\{l,r\}} - \boldsymbol{c}_{s,\{l,r\}}\|},$$

where $\|\cdot\|$ denotes the Euclidean norm.

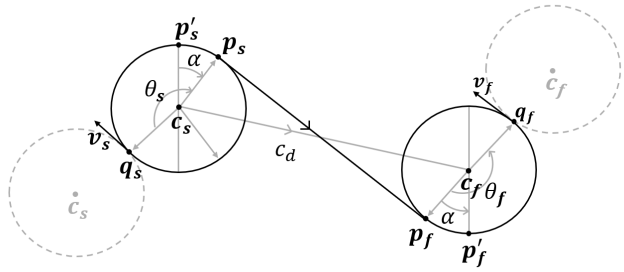


FIGURE 2. A graphical illustration of Dubins path generation in a 2D space.

Fig. 2 shows the RSL (Right-Straight-Left) case of the Dubins path, where the tangent points $\boldsymbol{p}_s$ and $\boldsymbol{p}_f$ between the circular arcs and the straight-line segment are computed as:

$$\boldsymbol{p}_{\{s,f\}} = \boldsymbol{c}_{\{s,f\}} + (\boldsymbol{p}'_s - \boldsymbol{c}_{\{s,f\}})\cos\alpha - \boldsymbol{n} \times (\boldsymbol{p}'_f - \boldsymbol{c}_{\{s,f\}})\sin\alpha,$$
$$\boldsymbol{p}'_s = \boldsymbol{c}_s + r(\boldsymbol{n} \times \boldsymbol{c}_d),$$
$$\boldsymbol{p}'_f = \boldsymbol{c}_f - r(\boldsymbol{n} \times \boldsymbol{c}_d),$$

where $\alpha = \sin^{-1}\left(\frac{2r}{\|\boldsymbol{c}_{r,f} - \boldsymbol{c}_{r,s}\|}\right)$ denotes the heading angle of the UAV. The rotational angles $\theta_{\{s,f\}}$, which represent the turning angles from the initial and target positions to their respective tangent points, can be defined as:

$$\theta_{\{s,f\}} = \begin{cases} \phi_{\{s,f\}} + \alpha, & \text{if } \boldsymbol{v}_{\{s,f\}} \cdot \boldsymbol{e}_{d,\{s,f\}} \geq 0 \\ 2\pi - \phi_{\{s,f\}} + \alpha, & \text{otherwise} \end{cases}, \quad (1)$$

where

$$\boldsymbol{e}_{d,\{s,f\}} = \frac{\boldsymbol{p}'_{\{s,f\}} - \boldsymbol{q}_{\{s,f\}}}{\left\|\boldsymbol{p}'_{\{s,f\}} - \boldsymbol{q}_{\{s,f\}}\right\|},$$
$$\phi_{\{s,f\}} = \cos^{-1}\left(\frac{(\boldsymbol{q}_{\{s,f\}} - \boldsymbol{c}_{\{s,f\}}) \cdot (\boldsymbol{p}'_{\{s,f\}} - \boldsymbol{c}_{\{s,f\}})}{r^2}\right).$$

The total length of a Dubins path is calculated as the sum of the initial circular arc, the straight-line segment, and the final circular arc, as follows:

$$l_{Dubins} = r\theta_s + \|\boldsymbol{p}_f - \boldsymbol{p}_s\| + r\theta_f. \quad (2)$$

Although the formulation is derived based on the RSL configuration, the same principle applies to the LSL, LSR, and RSR configuration as well. Among these four Dubins path types (RSL, LSL, LSR, RSR), the one with the shortest total length is selected as the final path.

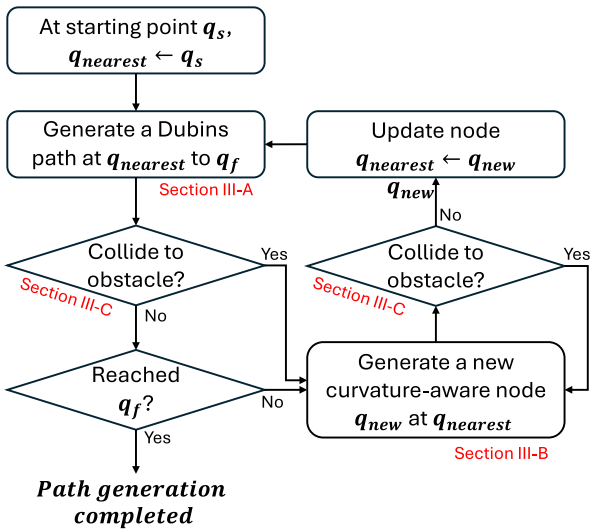


FIGURE 3. An overall procedure of the proposed DC-RRT* method.

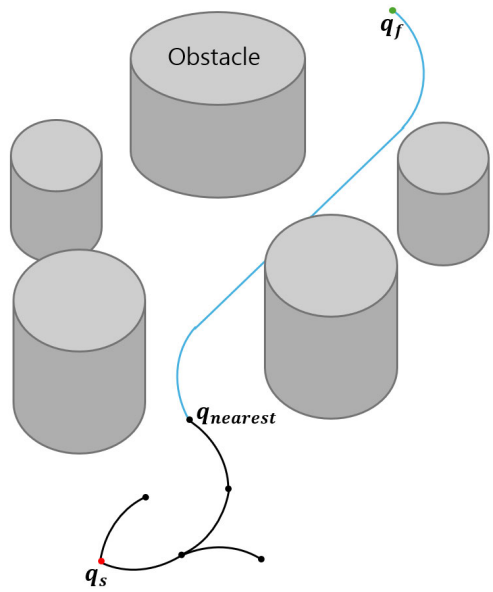


FIGURE 4. An illustration of Dubins path generation between $q_{nearest}$ and q_f in a 3D space where obstacles are scattered.

III. DC-RRT* FOR 3D UAV PATH PLANNING

Let $\boldsymbol{q}_s$ and $\boldsymbol{v}_s$ denote the coordinate and unit direction vector of the starting point, and $\boldsymbol{q}_f$ and $\boldsymbol{v}_f$ represent those of the target point, respectively. Here, each position coordinate vector $\forall \boldsymbol{q} = [x, y, z]^T$ and direction vector $\forall \boldsymbol{v} = [v_x, v_y, v_z]^T$. The primary objective of flight path generation is to compute a collision-free path from $\boldsymbol{q}_s$ to $\boldsymbol{q}_f$, which is a set of nodes $\boldsymbol{q}_i$ where $i = 1, \cdots$. Note that $\boldsymbol{q}_i = \boldsymbol{q}_s$ at the very first step of a path generation.

As illustrated in Fig. 3, the proposed DC-RRT* consists of three key components: i) Dubins path generation, ii) curvature-aware node generation, and iii) obstacle collision checking. As shown in the figure, we first generate a Dubins path connecting the current node (indicated by $\boldsymbol{q}_{nearest}$ in the figure) to the target point $\boldsymbol{q}_f$ (see Section III-A for details). If the generated path is confirmed to be free from collisions with any obstacles in the environment (Section III-C), the path generation process is considered complete. Note that unlike the conventional point-wise collision checking algorithm, our approach proposed in this work is based on volumes of UAVs.

On the contrary, if the generated Dubins path collides with any obstacle, the path generation is considered a failure. In such cases, the current path tree is expanded by adding a new node $\boldsymbol{q}_{new}$, which is generated using the proposed curvature-aware node generation algorithm (see Section III-B). Through this process, the resulting path is geometrically smooth and thus feasible for real-world applications without requiring any post-processing. Further details of the DC-RRT* method are provided in the following subsections.

A. DUBINS PATH GENERATION IN 3D SPACE

In this subsection, we extend the conventional Dubins path generation algorithm for 2D data which was discussed in Section II-B, to accommodate 3D data. The position of the UAV is expressed as a function of the curvilinear abscissa s as follows [31]:

$$\begin{aligned} \frac{dx(s)}{ds} &= \cos\gamma(s)\cos\psi(s), \\ \frac{dy(s)}{ds} &= \cos\gamma(s)\sin\psi(s), \\ \frac{dz(s)}{ds} &= \sin\gamma(s), \\ \frac{d\gamma(s)}{ds} &= \eta, \ \frac{d\psi(s)}{ds} = \mu, \end{aligned} \tag{3}$$

where γ and ψ denote the flight path angle and the heading angle of UAV, respectively. Both η and μ represent the rates of change in the flight path angle $\gamma(s)$ and the heading angle $\psi(s)$, respectively.

Given $(\boldsymbol{q}_s, \boldsymbol{v}_s)$ and $(\boldsymbol{q}_f, \boldsymbol{v}_f)$, the corresponding rotation centers $\boldsymbol{c}_s$ and $\boldsymbol{c}_f$ are computed as follows:

$$\boldsymbol{c}_{\{s,f\}} = \boldsymbol{q}_{\{s,f\}} + r \cdot R_z(\theta_{\{s,f\}}) \cdot \boldsymbol{v}_{\{s,f\}}, \tag{4}$$

where $R_z(\theta_{\{s,f\}}) \in \mathbb{R}^{3\times 3}$ denotes a rotation matrix about the vertical z-axis that rotates a vector by an angle $\theta_{\{s,f\}}$ within the $x-y$ plane. The tangent exit and entry points on the rotation circles are calculated as in the following:

$$\boldsymbol{p}_{\{s,f\}} = \boldsymbol{c}_{\{s,f\}} + r \cdot \begin{bmatrix} \cos\psi\cos\gamma \\ \sin\psi\cos\gamma \\ \sin\gamma \end{bmatrix}. \tag{5}$$

The total length l_{Dubins} of the Dubins path is calculated by summing the arc lengths and the straight-line segment, using the rotation angles θ_s and θ_f obtained following Eq.(1).

The four types of CSC path–LSL, LSR, RSL and RSR–each consisting of an initial circular arc, a line segment,

and a final circular arc, are evaluated by computing their respective path lengths. The path with minimum length l_{Dubins} is selected as the optimal path. This strategy is applied both during tree expansion and when attempting a direct connection to the target node upon the addition of a new node, thereby facilitating faster convergence to a feasible solution.

B. CURVATURE-AWARE NODE EXPANSION

As aforementioned, when path generation fails, a new curvature-aware node $\boldsymbol{q}_{new}$ is generated and added to the current path tree for expansion. As shown in Fig. 5 (a), a new node $\boldsymbol{q}_{new}$ of a path is generated by extending a distance ε from $\boldsymbol{q}_{nearest}$ toward a randomly sampled node $\boldsymbol{q}_{rand}$. To this end, $\boldsymbol{q}_{new}$ is redefined in the local coordinate frame relative to $\boldsymbol{q}_{nearest}$, such that $\boldsymbol{q}_{new} = [x_{rel}, y_{rel}]^T$.

To construct a curvature-constrained expansion path, as depicted in Fig. 5(b), we consider two spheres centered at $\boldsymbol{q}_{nearest}$ and $\boldsymbol{q}_{new}$, each with radius $r_\kappa = (x_{rel}^2 + y_{rel}^2)/(2y_{rel})$. The corresponding curvature is then defined as $\kappa = (2y_{rel})/(x_{rel}^2 + y_{rel}^2)$. These spheres intersect along a common arc, and the plane containing this intersection arc is defined as the reference plane for determining the center of rotation $\boldsymbol{o}_\kappa$. By projecting the node positions onto this plane, we compute the circular arc that satisfies the UAV's turning constraints.

The candidate rotation centers $\boldsymbol{o}_{\kappa_1}$ and $\boldsymbol{o}_{\kappa_2}$ are obtained as:

$$\boldsymbol{o}_{\kappa_1} = \boldsymbol{m} + h\vec{\boldsymbol{u}}, \quad \boldsymbol{o}_{\kappa_2} = \boldsymbol{m} - h\vec{\boldsymbol{u}},$$

where $\boldsymbol{m} = \boldsymbol{q}_{nearest} + \frac{1}{2}(\boldsymbol{q}_{new} - \boldsymbol{q}_{nearest})$ denotes the midpoint between $\boldsymbol{q}_{nearest}$ and $\boldsymbol{q}_{new}$, $h = \sqrt{r_\kappa^2 - \left(\frac{\varepsilon}{2}\right)^2}$ indicates the distance between $\boldsymbol{m}$ and each candidate center, and $\vec{\boldsymbol{u}}$ indicates the perpendicular bisector direction vector in the reference plane. For each candidate center, we compute the rotation angles required to connect $\boldsymbol{q}_{nearest}$ and $\boldsymbol{q}_{new}$ along a circular arc. The center that yields the smaller angular deviation in the intended direction of motion is selected as the final rotation center $\boldsymbol{o}_\kappa$. The circular trajectory is then generated using the Rodrigues rotation formula [32] as follows:

$$\vec{\boldsymbol{x}}(\theta) = \boldsymbol{o}_\kappa + (\vec{\boldsymbol{v}}_0 \cos\theta + (\vec{\boldsymbol{w}} \times \vec{\boldsymbol{v}}_0)\sin\theta + \vec{\boldsymbol{w}}(\vec{\boldsymbol{w}} \cdot \vec{\boldsymbol{v}}_0)(1 - \cos\theta)),$$

where $\vec{v}_0$ is the initial vector from the center of the arc to the starting point, and $\vec{w}$ is the unit axis of rotation, defined in accordance with the Rodrigues rotation formulation [32]. This expansion strategy ensures a smooth path composed of circular arcs, eliminating the need for additional smoothing in UAV trajectory tracking.

C. VOLUMETRIC COLLISION CHECK

RRT-based methods construct feasible paths by checking collisions during tree expansion, typically based on node-to-node or node-to-obstacle intersections. These approaches often rely on point-based collision checks, which are often insufficient to account for the actual volume and motion trajectory of a UAV. As a consequent, paths verified as collision-free in simulation may still result in collisions

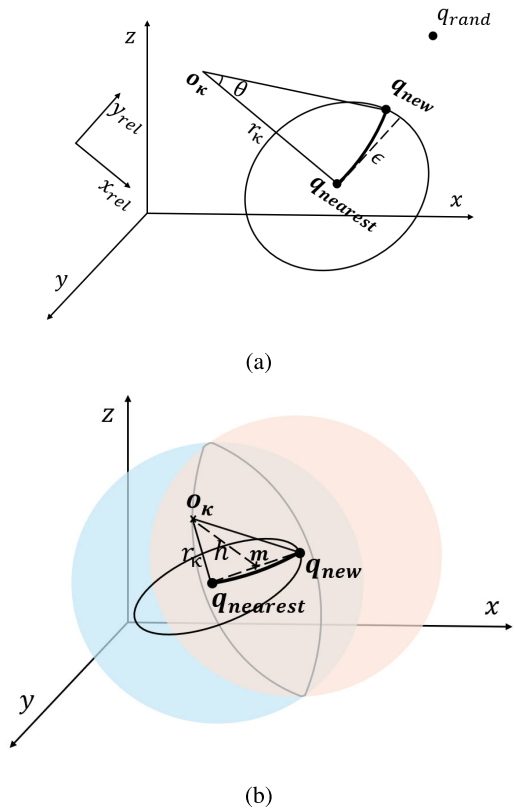


FIGURE 5. Generation of curvature-based path in (a) 2D space and (b) 3D space.

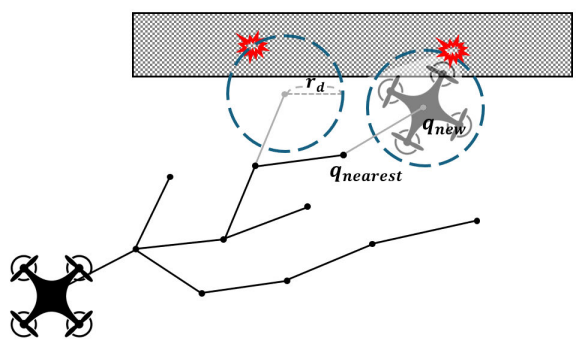


FIGURE 6. An illustration of spatial occupancy model for collision checking.

during real-world execution, particularly in complex and/or narrow environments, as illustrated in Fig. 6.

To address this issue, we propose a volumetric collision check that incorporates a ball-shaped spatial occupancy model to reflect the UAV's physical dimensions of radius r_d. In this model, the UAV is represented as a circular volume centered at its position, and the occupied region is defined as:

$$S_i = \{\boldsymbol{x} \mid \|\boldsymbol{x} - \boldsymbol{q}_i\| \leq r_d\}, \tag{6}$$

where $\boldsymbol{x} = [x, y, z]^T \in \mathbb{R}^3$ represents a point in space.

During tree expansion, when a new node $\boldsymbol{q}_{new}$ is generated, the method checks whether the corresponding occupancy region S_i intersects any obstacles. A collision is detected if the condition $S_i \cap \mathcal{O} \neq \emptyset$ is satisfied, where $\mathcal{O}$ denotes the set of obstacles defined in the environment. In such cases, the node is deemed invalid and excluded from the tree, thereby terminating path expansion at that point. This volumetric collision checking significantly enhances the reliability of path in confined environments compared to that of the conventional point-mass models [33], [34].

D. AN OPTIMALITY ANALYSIS OF DC-RRT*

1) LOCAL OPTIMALITY

DC-RRT* achieves local optimality through the incorporation of Dubins path, which inherently satisfy bounded curvature constraints. Dubins [28] demonstrated that the shortest feasible path connecting two configurations, subject to a maximum curvature constraint of $1/r$, comprises at most three segments, each being either a straight line or a circular arc. These paths also exhibit Lipschitz continuity. In DC-RRT*, each newly generated node $\boldsymbol{q}_{new}$ attempts a direct Dubins-path connection to the target node $\boldsymbol{q}_f$. If this path proves valid and collision-free, the node is immediately integrated into the tree, thereby facilitating locally optimal tree expansion.

2) GLOBAL ASYMPTOTIC OPTIMALITY

RRT* guarantees asymptotic optimality provided two conditions are met: uniform sampling of the configuration space and a Lipschitz-continuous steering function [15]. DC-RRT* maintains uniform sampling, thus satisfying the first condition. Regarding the second, DC-RRT* preserves the structure of RRT* while replacing the steering function with a curvature-aware mechanism. Particularly, DC-RRT* connects nodes using the shortest feasible Dubins path whenever available. If such a path does not exist, the algorithm employs curvature-aware node expansion. Since this curvature-aware expansion defines a valid steering function for all sampled nodes and ensures continuous variation in path cost, it inherently satisfies Lipschitz-continuity. Consequently, DC-RRT* meets both optimality conditions, thereby ensuring global asymptotic optimality.

E. SUMMARY

The detailed procedure of DC-RRT* is presented in Algorithm 1.

Algorithm 1 DC-RRT*

Input: Start position $\boldsymbol{q}_s$, goal position $\boldsymbol{q}_f$, threshold $R_{\text{threshold}}$, step size ϵ, turning radius r
Output: Path from $\boldsymbol{q}_s$ to $\boldsymbol{q}_f$
Initialize tree T with $\boldsymbol{q}_{nearest} \leftarrow \boldsymbol{q}_s$
while True **do**
  $\text{path}_{\text{dubins}} \leftarrow \text{GenerateDubinsPath}(\boldsymbol{q}_{nearest}, \boldsymbol{q}_f, r)$
  if not $\text{CollisionCheck}(\text{path}_{\text{dubins}}, \mathcal{S}_i)$ **then**
    if $\text{ReachedGoal}(\boldsymbol{q}_{nearest}, \boldsymbol{q}_f, R_{\text{threshold}})$ **then**
      return $\text{ExtractPath}(T, \boldsymbol{q}_s, \boldsymbol{q}_f)$
    end if
  else
    $\boldsymbol{q}_{rand} \leftarrow \text{RandomSample}()$
    $\boldsymbol{v} \leftarrow \text{Direction}(\boldsymbol{q}_{nearest}, \boldsymbol{q}_{rand})$
    $\boldsymbol{q}_{tent} \leftarrow \boldsymbol{q}_{nearest} + \epsilon \cdot \boldsymbol{v}$
    $\boldsymbol{q}_{rel} \leftarrow \text{TransformToLocalFrame}(\boldsymbol{q}_{nearest}, \boldsymbol{q}_{tent})$
    $\kappa,\ r_\kappa \leftarrow \text{ComputeCurvature}(\boldsymbol{q}_{rel})$
    $\boldsymbol{o}_\kappa \leftarrow \text{SelectRotationCenter}(\boldsymbol{q}_{nearest}, \boldsymbol{q}_{tent}, r_\kappa)$
    $\boldsymbol{q}_{new} \leftarrow \text{InterpolateAlongArc}(\boldsymbol{q}_{nearest}, \boldsymbol{o}_\kappa, \epsilon)$
    if $\text{CollisionCheck}(\boldsymbol{q}_{nearest}, \boldsymbol{q}_{new}, \mathcal{S}_i)$ **then**
      continue
    end if
    Add $\boldsymbol{q}_{new}$ to tree T with parent $\boldsymbol{q}_{nearest}$
    $\boldsymbol{q}_{nearest} \leftarrow \boldsymbol{q}_{new}$
  end if
end while

IV. SIMULATION STUDY

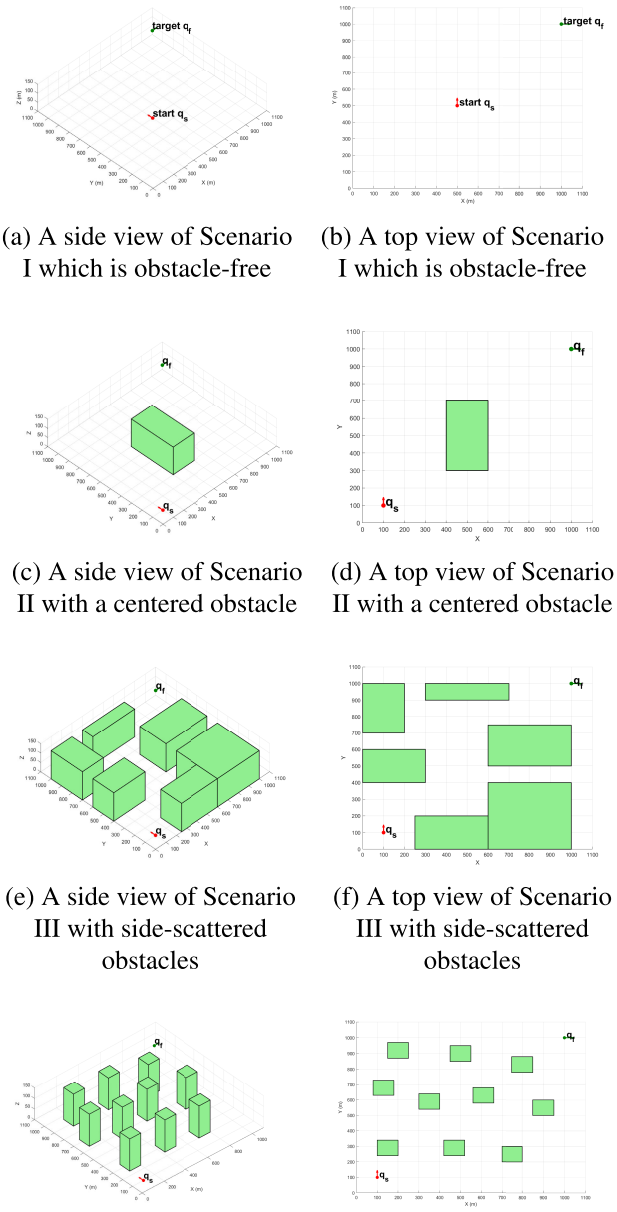
A. EVALUATION PROTOCOLS AND SCENARIOS

To evaluate the path generation performance of the proposed DC-RRT*, we set up simulation environments in a three-dimensional space of size 1000m×1000m×150m following the setup in [31]. We consider four scenarios with different obstacle configurations: i) an obstacle-free scenario (see Fig.7 (a) and (b)), ii) a centered obstacle scenario (Fig.7 (c) and (d)), iii) a side-scattered obstacle scenario (Fig.7 (e) and (f)), and lastly iv) a densely-scattered obstacle scenario (Fig.7 (g) and (h)). The first three configurations are taken from [31], while the last challenging configuration is taken from [24]. As shown in Table 1, we call the scenarios respectively as Scenario I, II, III and IV hereafter for convenience. The primary objective of having these scenarios is to evaluate the performance of the proposed and benchmarking methods under different environmental conditions.

TABLE 1. Four simulation scenarios and environmental configurations.

Sce. No.	Brief descriptions	Dimension $[H, W, Z]$
I	Generating paths for a 3D space with no obstacle. The simplest scenario	[1000, 1000, 150] meters
II	Generating paths for a 3D simulation space where an obstacle is located at the center	
III	Generating paths for a 3D simulation space where multi obstacles are side-scattered	
IV	Generating paths for a 3D simulation space where randomly generated dense obstacles are scattered	

As shown in Fig. 7, the primary task of the scenarios is to generate flying paths from a starting point $\boldsymbol{q}_s$ to a target point $\boldsymbol{q}_f$. A trial of path generation is regarded successful if the $\boldsymbol{q}_f$ is reached within a distance threshold, $R_{\text{threshold}}$. If the $\boldsymbol{q}_f$ is not reached within $R_{\text{threshold}}$ or the number of tree nodes exceeds 1000, the trial is regarded as failure [31]. Parameters ϵ, $R_{\text{threshold}}$ and R_n are respectively set to 30m, 50m and



(a) A side view of Scenario I which is obstacle-free

(b) A top view of Scenario I which is obstacle-free

(c) A side view of Scenario II with a centered obstacle

(d) A top view of Scenario II with a centered obstacle

(e) A side view of Scenario III with side-scattered obstacles

(f) A top view of Scenario III with side-scattered obstacles

(g) A side view of Scenario IV with dense-obstacles

(h) A top view of Scenario IV with dense-obstacles

FIGURE 7. Graphical illustrations of the simulated 3D environments.

$\epsilon \times$ 8m according to [31]. Based on [27], the goal bias probability k was set at 10%, and the minimum turning radius r of the proposed method was defined as 80m as described in [31]. Table 2 tabulates coordinates of both $\boldsymbol{q}_{\{s,f\}}$ and $\boldsymbol{v}_{\{s,f\}}$ utilized in generating the environments.

To position the proposed DC-RRT* among existing methods in terms of path generation performance, eight benchmarking methods–RRT [13], RRT* [15], T-RRT* [35], F-RRT* [34], PF-RRT* [18], CAF-RRT* [26], Bi-APF-RRT* [25], and the analytical continuous-curvature path-smoothing (ACPS) [36]–are included in our simulation and comparison study. Except for the baseline RRT method, the remaining methods consists of the conventional RRT* and its variants proposed in recent years. It is noteworthy that CAF-RRT* aims to produce smooth trajectories while ACPS employ curvature-based smoothing to generate dynamically feasible paths. All of these methods were selected based on the availability of publicly accessible source code. Other relevant methods for which source codes are not available were excluded from the comparison to ensure fairness.

TABLE 2. Waypoints for simulation.

Scenarios	Start point, $\boldsymbol{q}_s$	Target point, $\boldsymbol{q}_f$	Directions $\boldsymbol{v}_s$	Directions $\boldsymbol{v}_f$
Scenario I	$\begin{bmatrix}500\\500\\1\end{bmatrix}$	$\begin{bmatrix}1000\\1000\\100\end{bmatrix}$	$\begin{bmatrix}0\\1\\0\end{bmatrix}$	$\begin{bmatrix}1\\0\\0\end{bmatrix}$
Scenario II Scenario III Scenario IV	$\begin{bmatrix}100\\100\\1\end{bmatrix}$			

For performance evaluation, both quantitative and qualitative analyses are conducted. For the quantitative analysis, three performance metrics are considered [31]: i) the number of nodes per path, ii) the path length (in meters) and iii) the CPU time (in seconds) elapsed for generating a path. For each metric, lower values indicate better performance. In the qualitative analysis, the paths generated by each method are visually investigated as illustrated in Figs. 8-11. To account for statistical evidence over the randomness in $\boldsymbol{q}_{rand}$, the DC-RRT* is run 100 times where the average performance is recorded. All simulations were performed using MATLAB R2024b on a computer equipped with a 3.60 GHz CPU and 32 GB RAM.

B. SCENARIO I. OBSTACLE-FREE ENVIRONMENT

Table 3 shows the average results of all evaluated methods across three performance metrics: the number of nodes per path, path length and CPU time, taken over 100 iterations. To highlight the best and second-best performing methods, the lowest recorded result in each category is shown in bold while the second-lowest is underlined. It is evident from Table 3 that the proposed DC-RRT* outperforms the compared methods in terms of the number of nodes and CPU time. Particularly, DC-RRT* requires just two nodes on average, which is about 22 times lower than the second-best method, CAF-RRT*. Compared to DC-RRT*, the conventional RRT* generates about 70 times more nodes. Among all evaluated methods, ACPS requires the highest number of nodes.

A smaller number of nodes generally indicates faster convergence during path generation. This trend is further supported by the CPU time results shown in the right-most column of Table 3. Both DC-RRT* and CAF-RRT*, which yield the fewest nodes, require the lowest and second-lowest CPU times, respectively. Notably, DC-RRT* achieves nearly twice the speed of CAF-RRT*. Among the evaluated methods, RRT* appears to be the least efficient in terms of CPU time.

TABLE 3. Quantitative analysis results for Scenario I, with all values reported as the mean (±standard deviation) over 100 iterations.

Methods	Number of nodes	Path length (m)	CPU time (s)
RRT	313.43(±125.83)	876.27(±52.52)	0.1748(±0.1359)
RRT*	151.04(±68.72)	709.92(±3.53)	0.2210(±0.2180)
ACPS	286.72(±108.98)	783.12(±32.85)	0.2010(±0.1096)
T-RRT*	145.26(±51.74)	709.67(±2.79)	0.2021(±0.1506)
F-RRT*	159.42(±36.48)	**708.54(±2.17)**	0.2141(±0.0929)
PF-RRT*	138.06(±33.37)	708.65(±2.25)	0.1644(±0.0752)
CAF-RRT*	44.33(±7.99)	725.9(±14.84)	0.0115(±0.0050)
Bi-APF-RRT*	67.96(±11.57)	717.60(±10.18)	0.0292(±0.0093)
DC-RRT*	**2.00(±0.00)**	719.62(±0.00)	**0.0069(±0.0003)**

TABLE 4. Quantitative analysis results for Scenario II, with all values reported as the mean (±standard deviation) over 100 iterations.

Methods	Number of nodes	Path length (m)	CPU time (s)
RRT	685.47(±158.87)	1715.45(±80.78)	0.8282(±0.3540)
RRT*	462.03(±187.21)	1358.79(±10.34)	2.3278(±1.9012)
ACPS	694.31(±150.97)	1528.90(±101.32)	1.1121(±0.3828)
T-RRT*	182.99(±48.73)	1411.42(±32.28)	0.3583(±0.1929)
F-RRT*	336.36(±63.32)	**1343.94(±1.73)**	1.6975 (±0.5321)
PF-RRT*	291.28(±54.55)	1344.05(±3.03)	1.3541(±0.5071)
CAF-RRT*	121.48(±22.94)	1399.68(±30.33)	0.1147(±0.0380)
Bi-APF-RRT*	265.40(±49.10)	1359.74(±14.07)	0.5096(±0.1965)
DC-RRT*	**52.45(±21.70)**	1390.98(±35.40)	**0.1058(±0.0214)**

Unlike these two metrics discussed above, the path length of DC-RRT* falls around the average range among the compared methods. The top four methods–PF-RRT*, F-RRT*, T-RRT* and RRT*–form the best-performing group for path length, while Bi-APF-RRT*, DC-RRT* and CAF-RRT* comprise the second-best performing group. It is worth noting that the average difference between the two groups is marginal.

Fig. 8 presents, as for the qualitative results, spatial distributions of 100 paths generated by the evaluated methods. As shown in Fig. 8 (a) and (c), the paths generated by RRT and ACPS exhibit non-smooth and zigzagging behavior, most likely due to local randomness. Note that this trend of results aligns with the higher number of nodes generated. Paths generated by the other remaining methods (Fig. 8 (b), (d)-(i)) seem much stabler than those of the RRT and the ACPS.

Particularly, the proposed DC-RRT* in Fig. 8 (i) outperforms the compared methods in terms of path consistency and geometric regularity. All paths converge precisely toward the target with minimal deviation, and are straight and smooth, thanks to the curvature-aware node expansion and goal-directed biasing strategies. Notably, the path shapes are free from oscillations or abrupt turns, which are commonly observed from the results of RRT, ACPS, CAF-RRT* and Bi-APF-RRT*. These results confirm that DC-RRT* achieves stable performance in both path quality and computational speed by effectively reducing unnecessary exploration in simple, obstacle-free environments.

C. SCENARIO II. CENTERED-OBSTACLE ENVIRONMENT

In this scenario, we evaluate the effectiveness of each method in a more constrained setting. As shown in Fig. 7 (c) and (d), a rectangular obstacle is placed between the start point $\boldsymbol{q}_s$ and the target point $\boldsymbol{q}_f$, forcing each method to find a feasible path around the obstacle while maintaining efficiency.

Similar to Table 3, averages over 100 trials for each method are tabulated in Table 4. DC-RRT* again demonstrates superior performance in terms of the number of nodes and CPU time. On average, it generates valid paths using only around 52 nodes, significantly overwhelming all the compared methods. For instance, the second-best method, CAF-RRT* requires around twice more nodes while the remaining methods require around 3.5 to 13 times more nodes than that of the DC-RRT*. Furthermore, the proposed DC-RRT* reveals competitive CPU time performance, about 0.1058 seconds on average per trial, which is slightly lower than that of CAF-RRT* (0.1147 seconds), yet considerably much lower than all other methods.

In terms of path length, similar to the results of Scenario I, F-RRT* and PF-RRT* methods again achieve the shortest path. It is worth noting here that, however, the difference in path length between the best-performing group (i.e., F-RRT* and PF-RRT*) and DC-RRT* is marginal. It becomes much clearer when we consider the qualitative results together as shown in Fig. 9.

As depicted in the sub-figures Fig. 9 (a)-(h), the paths generated by the compared methods closely follow the boundaries of obstacle. This is because the compared methods do not account the physical parameters of UAVs such as turning radius and body size. In such cases, the paths may lead UAVs to potential collisions during real-world operations. On the contrary, as shown in Fig. 9 (i), the generated paths by DC-RRT* are smooth and comply the curvature property making it more suitable for deployment in real-world UAV flights. Moreover, additional post-processing such as path smoothing is unnecessary in the proposed method.

D. SCENARIO III. SIDE-SCATTERED-OBSTACLES ENVIRONMENT

This third scenario considers an environment with multiple side-scattered obstacles (see Fig. 7 (e) and (f)). The spatial distribution of obstacles increases the likelihood of collisions, thereby imposing a more complex path-planning condition. Similar to the results observed in Scenarios I and II, Table 5 indicates that the proposed DC-RRT* and CAF-RRT* methods again achieve the best and second-best performance, respectively, in terms of node count. However, in terms of path length, the proposed method performs slightly worse than the other compared methods, except for RRT and ACPS. Nevertheless, the difference is marginal.

As shown in Fig. 10 (a)-(h), such performance trend can be attributed to the preference of the compared methods

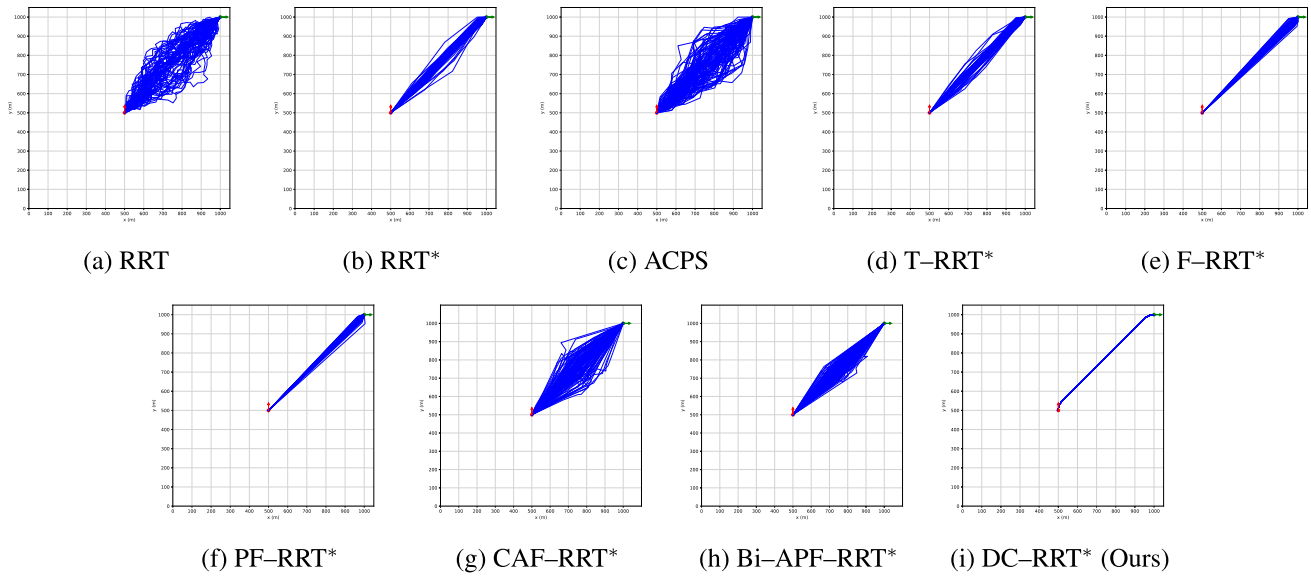


FIGURE 8. Top views of qualitative analysis results obtained over 100 trials under Scenario I, where no obstacle exists in the simulated 3D space. Note that a single solid line represents the result of a path generation trial.

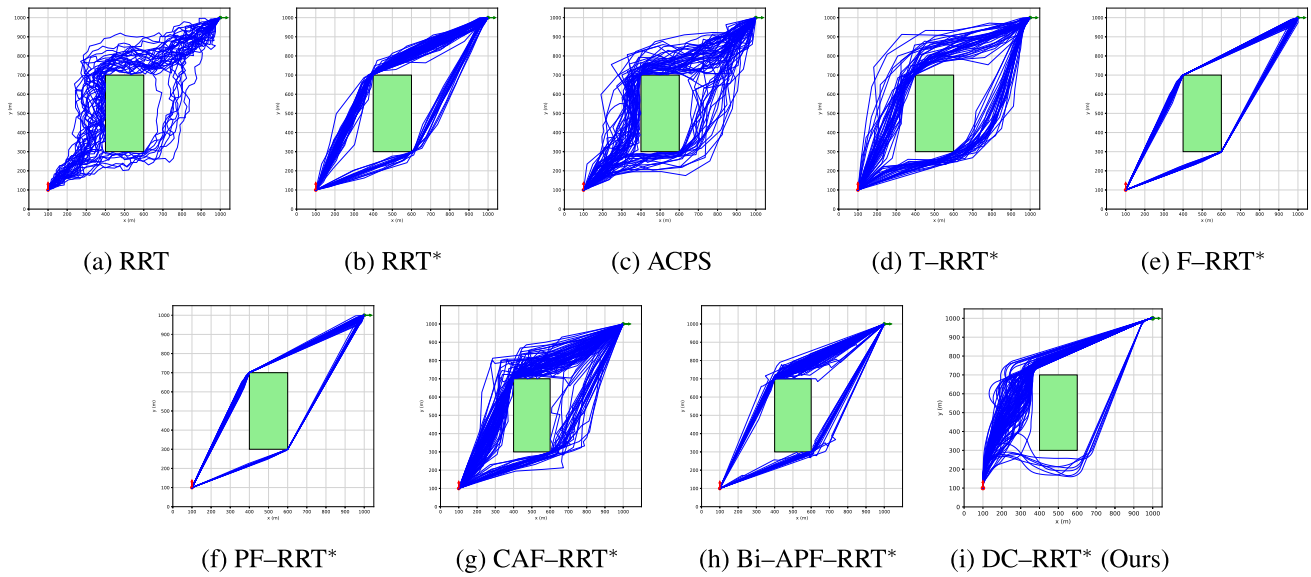
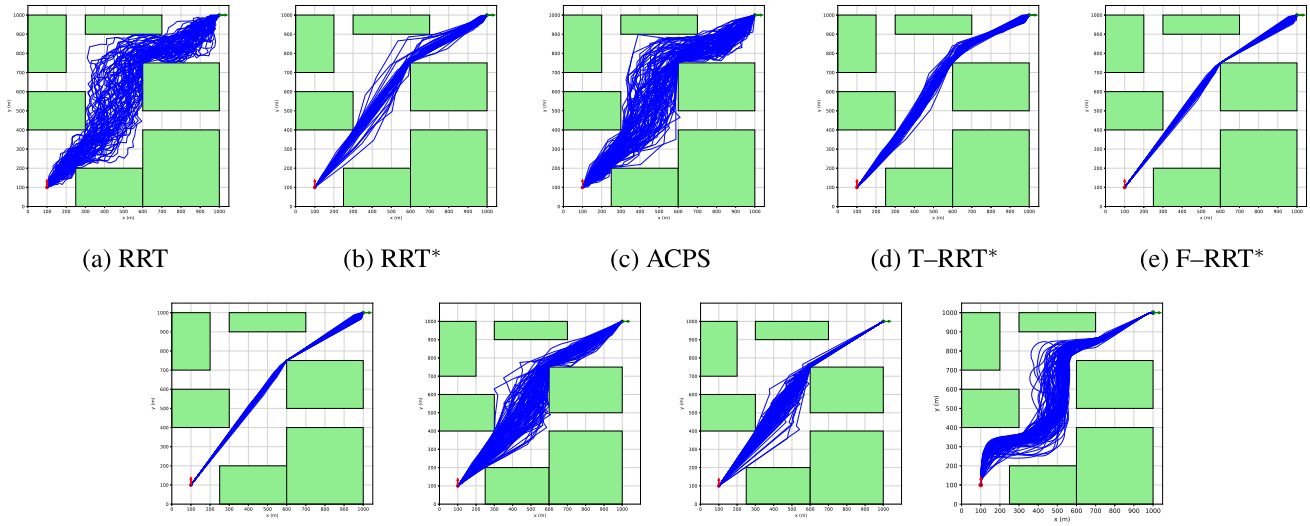


FIGURE 9. Top views of qualitative analysis results obtained over 100 trials under Scenario II, where an obstacle is located at the center of the simulated 3D space.

for connecting nodes using straight-line segments. Moreover, the resulting paths often run very close to boundaries of obstacles. When considering real-world environments, it becomes evident that such straight-line paths fail to account for both the physical dimensions and the maneuverability constraints of UAVs, rendering them infeasible for practical deployment. Although the paths generated by DC-RRT* are slightly longer, the algorithm accounts for these constraints during the planning stage and thus produces collision-free paths without requiring any post-processing.

E. SCENARIO IV. DENSELY SCATTERED-OBSTACLES ENVIRONMENT

This last scenario, characterized by densely scattered obstacles throughout the environment, represents the most challenging configuration. As shown in Fig. 7 (g) and (h), the densely scattered obstacles create narrow passages, significantly increasing the risk of collisions during UAV maneuvering. According to the numerical results presented in Table 6, similar to the results observed in Scenarios I–III, the proposed DC-RRT* and CAF-RRT* achieve the best and



(a) RRT (b) RRT* (c) ACPS (d) T–RRT* (e) F–RRT*

(f) PF–RRT* (g) CAF–RRT* (h) Bi–APF–RRT* (i) DC–RRT* (Ours)

FIGURE 10. Top views of qualitative analysis results obtained over 100 trials under Scenario III, where obstacles are scattered across the simulated 3D space.

TABLE 5. Quantitative analysis results for Scenario III, with all values reported as the mean (±standard deviation) over 100 iterations.

Methods	Number of nodes	Path length (m)	CPU time (s)
RRT	628.42(±180.76)	1676.52(±88.64)	1.1792(±0.6380)
RRT*	349.51(±123.93)	1300.82(±8.00)	3.6794(±2.7481)
ACPS	581.31(±172.99)	1466.80(±86.04)	2.1713(±0.9395)
T-RRT*	129.87(±49.99)	1309.03(±8.59)	0.9200(±0.9769)
F-RRT*	322.24(±59.57)	**1292.73(±1.01)**	2.3942 (±0.8401)
PF-RRT*	304.98(±75.02)	1292.78(±1.21)	2.1835(±1.1555)
CAF-RRT*	85.60(±15.45)	1352.67(±43.42)	**0.2258(±0.0612)**
Bi-APF-RRT*	162.60(±29.58)	1307.47(±11.19)	0.3343(±0.1243)
DC-RRT*	**66.57(±13.80)**	1410.49(±44.60)	0.7839(±0.2657)

TABLE 6. Quantitative analysis results for Scenario IV, with all values reported as the mean (±standard deviation) over 100 iterations.

Methods	Number of nodes	Path length (m)	CPU time (s)
RRT	764.50(±145.22)	1621.50(±95.85)	1.1804(±0.3902)
RRT*	448.88(±151.13)	1293.48(±8.55)	7.4260(±4.9570)
ACPS	735.46(±156.45)	1435.28(±73.71)	4.0874(±2.0904)
T-RRT*	138.52(±59.92)	red1365.87(±29.18)	1.1746(±1.3269)
F-RRT*	401.40(±100.98)	**1288.25(±12.42)**	4.3037 (±2.0570)
PF-RRT*	373.84(±133.32)	1292.78(±9.55)	2.1835(±2.7864)
CAF-RRT*	89.65(±16.94)	1337.54(±39.85)	**0.3472(±0.1050)**
Bi-APF-RRT*	157.76(±36.33)	1299.63(±12.28)	0.4732(±0.2087)
DC-RRT*	**87.24(±15.90)**	1421.34(±39.20)	0.9135(±0.4090)

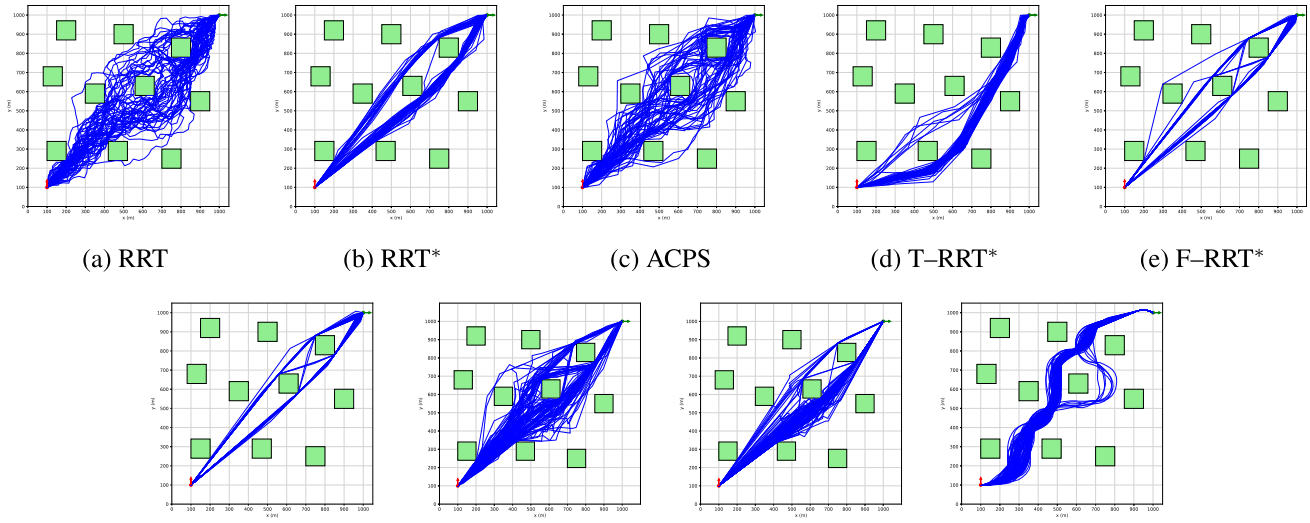
second-best performance in terms of node count. Regarding computational complexity, CAF-RRT* and Bi-APF-RRT* demonstrate the fastest performance due to their bidirectional connections, followed by the proposed DC-RRT*.

As shown in Fig.11, and consistent with the results in Figs.8–10, the paths generated by the compared methods largely consist of straight-line segments that closely approach obstacles. In contrast, DC-RRT* produces smooth and continuous paths that maintain sufficient clearance by considering curvature constraints during the planning phase, thereby eliminating the need for any post-processing.

F. AN EMPIRICAL ANALYSIS ON PARAMETER SETTINGS

In this empirical analysis, we investigate the impact of two key parameters: the UAV's turning radius r and goal biasing probability k, on the path generation performance of DC-RRT*. Table 7 summarizes the three evaluation metrics across various radius settings $r = \{40, 60, \cdots, 140\}$ under the four Scenarios. The results indicate notable differences between Scenarios I-II and Scenarios III-IV. Specifically, in Scenario I and II, characterized by simpler obstacle configurations, the number of nodes and CPU time either remain unchanged or decrease with increasing r, while path length generally increases. In contrast, Section III and IV, featuring densely cluttered obstacles, exhibit a significant increase in both node count and CPU time as r grows from 120 to 140. These results highlight that a larger turning radius restricts UAV maneuverability within narrow passages, consequently elevating collisions risk.

Similar to Table 7, Table 8 presents the three evaluation metrics across varying goal biasing probability $k = \{5, 10, \cdots, 30\}$ under Scenarios II, III and IV. Under Scenario I, DC-RRT* immediately reaches the target point; therefore, the results are unaffected by the choice of k. As indicated in Table 8, the optimal performance is achieved at $k = 15$ for Scenario II and at $K = 20$ under



(a) RRT (b) RRT* (c) ACPS (d) T–RRT* (e) F–RRT*

(f) PF–RRT* (g) CAF–RRT* (h) Bi–APF–RRT* (i) DC–RRT* (Ours)

FIGURE 11. Top views of qualitative analysis results obtained over 100 trials under Scenario IV, where obstacles are clustered the simulated 3D space.

TABLE 7. Performance metrics across turning radius r for each scenario.

Sce. No.	Measures	Turning radius r (m)					
		40	60	80	100	120	140
I	Node count	2.00	2.00	2.00	2.00	2.00	2.00
	Path length	713.37	716.50	719.62	722.75	725.88	729.01
	CPU time	0.0069	0.0071	0.0069	0.0069	0.0068	0.0070
II	Node count	58.68	55.67	52.45	51.20	49.76	49.07
	Path length	1387.4	1388.22	1390.98	1403.48	1412.09	1419.18
	CPU time	0.1196	0.1150	0.1058	0.1030	0.1028	0.1019
III	Node count	70.91	67.65	66.57	62.86	58.62	77.41
	Path length	1407.47	1408.97	1410.49	1422.52	1431.29	1443.47
	CPU time	1.3810	1.1990	0.7839	0.6767	0.4916	1.3810
IV	Node count	92.73	90.16	87.24	81.87	81.74	105.16
	Path length	1412.82	1417.38	1421.34	1434.52	1439.96	1455.40
	CPU time	1.1203	1.1061	0.9135	0.7601	0.7123	1.2061

TABLE 8. Performance metrics across goal biasing probability k for each scenario.

Sce. No.	Measures	Goal biasing probability k (%)					
		5	10	15	20	25	30
II	Node count	57.97	52.45	52.35	52.69	53.35	56.62
	Path length	1400.40	1390.98	1388.07	1392.07	1394.33	1399.22
	CPU time	0.1336	0.1058	0.1055	0.1075	0.1099	0.1250
III	Node count	70.18	66.57	65.61	64.87	66.98	67.84
	Path length	1422.23	1410.49	1409.28	1405.48	1410.05	1413.04
	CPU time	0.8365	0.7839	0.7827	0.7757	0.7981	0.7981
IV	Node count	97.25	87.24	86.28	84.95	88.63	92.99
	Path length	1438.87	1421.34	1419.62	1417.23	1420.65	1426.73
	CPU time	0.9836	0.9135	0.9078	0.8929	0.9183	0.9546

Scenarios III and IV. While certain trends are evidence across different K settings–for example, in Scenario III, all metrics improve with increasing k until reaching the optimal value at $k = 20$, after which performances deteriorates–the observed differences among k settings appear marginal.

TABLE 9. Average CPU time (s) per component across scenarios.

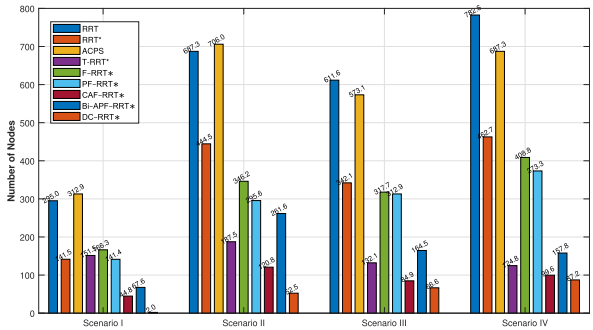
Sce. No.	Step i) Dubins path generation	Step ii) Curvature node expansion	Step iii) Collision check in step i)	Step iv) Collision check in step ii)	Total time
I	0.0040	0.0000	0.0028	0.0000	0.0069
II	0.0226	0.0331	0.0305	0.0170	0.1058
III	0.1921	0.2344	0.2148	0.1191	0.7839
IV	0.2124	0.2817	0.2437	0.1437	0.9135

G. AN ANALYSIS ON CPU TIME PERFORMANCE

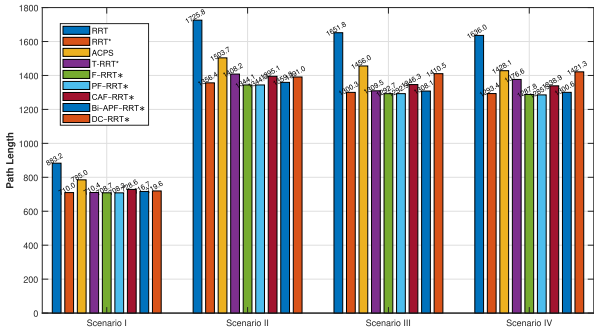
As shown in Table 9, a component-wise runtime analysis is conducted to assess the scalability of the proposed DC-RRT*. Particularly, we measure the elapsed CPU time of the four primary steps: i) Dubins path generation (see Section III-A), ii) curvature-based node expansion (see Section III-B), and checking collisions (see Section III-C) during iii) the path generation, and iv) the node expansions. The results shown in Table 9 indicate that as density and complexity of obstacles grow (i.e., from Scenario I to IV), the unsuccessful rate of Dubins path generation increases. Consequently, the DC-RRT* relies more on curvature-aware node expansions, incurring additional computational overhead.

H. RESULTS SUMMARY AND DISCUSSIONS

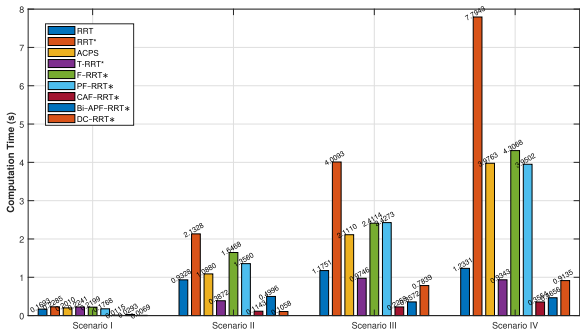
Fig. 12 provides a summary of the entire results across all four scenarios. As observed from Fig. 12, the proposed DC-RRT* consistently outperformed or matched the benchmarking methods across all scenarios. In both the obstacle-free and centered-obstacle environments, DC-RRT* achieved the lowest node count and CPU time while maintaining competitive path lengths. In the last two scenarios



(a) Average number of nodes



(b) Average path length



(c) Average CPU time

FIGURE 12. A summary of the entire quantitative results for comparison across scenarios.

where obstacles are scattered throughout the environment, DC-RRT* continued to demonstrate superior node efficiency and generated smooth, collision-free paths. However, as shown in Fig. 12 (c) and discussed in Section IV-G, the CPU time increased as the density and complexity of obstacles grow. This increase primarily resulted from failures in Dubins path generation, necessitating curvature-aware node expansions.

Although some compared methods yielded slightly shorter paths, they often produced paths that run dangerously close to obstacles due to their reliance on straight-line connections and lack of curvature constraints. In contrast, DC-RRT* explicitly incorporated UAV maneuverability and curvature constraints, ensuring more realistic and deployable paths without requiring additional post-processing. Overall, DC-RRT* achieved a strong balance between planning efficiency, path quality, and real-world feasibility, making it a practical and robust solution for UAV path planning in diverse environments.

V. CONCLUSION AND FUTURE WORKS

In this paper, we proposed DC-RRT*, a novel path planning algorithm for UAVs that explicitly accounted for the physical constraints of UAVs such as the minimum turning radius and body dimensions. To this end, DC-RRT* employed a curvature-aware node expansion strategy that incorporated both heading direction and turning constraints. It then enhanced convergence by attempting Dubins path connections between newly generated nodes and the goal. Our simulation study, which quantitatively and qualitatively compared the performance of DC-RRT* with eight existing methods, demonstrated that DC-RRT* significantly outperformed the compared methods in terms of node efficiency. Although the paths generated by DC-RRT* were slightly longer in certain scenarios, they were more realistic and feasible, particularly for real-world UAV operations.

Note that DC-RRT* was primarily evaluated within simulation environments containing static obstacles and no additional moving objects. Nevertheless, based on our results, the computational complexity of the proposed method appears acceptable for real-time, online path planning. As a next step, we aim to adapt and extend DC-RRT* explicitly for online planning scenarios. Moreover, future research directions will include onboard implementation on actual UAV hardware, as well as comprehensive performance evaluation in dynamic environments involving multiple UAVs, wind disturbances, and perception uncertainties.

REFERENCES

[1] S. Kim, D. Kim, S. Jeong, J.-W. Ham, J.-K. Lee, and K.-Y. Oh, "Fault diagnosis of power transmission lines using a UAV-mounted smart inspection system," *IEEE Access*, vol. 8, pp. 149999–150009, 2020.

[2] L. Chen, X. Pang, Z. Wu, R. Huang, J. Hu, Y. Liu, L. Zhou, J. Zhou, and Z. Wang, "Unmanned aerial vehicles equipped with sensor packages to study spatiotemporal variations of air pollutants in industry parks," *Phil. Trans. Roy. Soc. A: Math., Phys. Eng. Sci.*, vol. 382, no. 2281, Oct. 2024, Art. no. 20230314.

[3] T. Calamoneri, F. Corò, and S. Mancini, "Management of a post-disaster emergency scenario through unmanned aerial vehicles: Multi-depot multi-trip vehicle routing with total completion time minimization," *Expert Syst. Appl.*, vol. 251, Oct. 2024, Art. no. 123766.

[4] A. Brown and D. Anderson, "Trajectory optimization for high-altitude long-endurance UAV maritime radar surveillance," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 56, no. 3, pp. 2406–2421, Jun. 2020.

[5] M. A. Istiak, M. M. M. Syeed, M. S. Hossain, M. F. Uddin, M. Hasan, R. H. Khan, and N. S. Azad, "Adoption of unmanned aerial vehicle (UAV) imagery in agricultural management: A systematic literature review," *Ecol. Informat.*, vol. 78, Dec. 2023, Art. no. 102305.

[6] M. Tariq, A. Saadat, R. Ahmad, Z. Abaid, and J. J. P. C. Rodrigues, "Enhanced border surveillance through a hybrid swarm optimization algorithm," *IEEE Sensors J.*, vol. 23, no. 22, pp. 28172–28181, Nov. 2023.

[7] J. Muñoz, B. López, F. Quevedo, C. A. Monje, S. Garrido, and L. E. Moreno, "Multi UAV coverage path planning in urban environments," *Sensors*, vol. 21, no. 21, p. 7365, Nov. 2021.

[8] G. Aiello, K. P. Valavanis, and A. Rizzo, "Fixed-wing UAV energy efficient 3D path planning in cluttered environments," *J. Intell. Robotic Syst.*, vol. 105, no. 3, p. 60, Jul. 2022.

[9] H. Shakhatreh, A. H. Sawalmeh, A. Al-Fuqaha, Z. Dou, E. Almaita, I. Khalil, N. S. Othman, A. Khreishah, and M. Guizani, "Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges," *IEEE Access*, vol. 7, pp. 48572–48634, 2019.
[10] A. Stentz, "Optimal and efficient path planning for unknown and dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom.*, CA, CA, USA, May 1993, pp. 3310–3317.
[11] P. Bogiatzis, C. A. Rychert, and N. Harmon, "Multiple graph realizations method: Improving the accuracy and the efficiency of the shortest path method through random sampling," *Geophys. J. Int.*, vol. 227, no. 1, pp. 669–679, May 2021.
[12] S. J. Anderson, S. C. Peters, T. E. Pilutti, and K. Iagnemma, "An optimal-control-based framework for trajectory planning, threat assessment, and semi-autonomous control of passenger vehicles in hazard avoidance scenarios," *Int. J. Vehicle Auto. Syst.*, vol. 8, no. 2, pp. 190–216, 2010.
[13] S. Lavalle, "Rapidly-exploring random trees: A new tool for path planning," Tech. Rep. 9811, 1998.
[14] J. Wang, W. Chi, C. Li, C. Wang, and M. Q.-H. Meng, "Neural RRT: Learning-based optimal path planning," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 4, pp. 1748–1758, Oct. 2020.
[15] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, Jun. 2011.
[16] I. Noreen, A. Khan, H. Ryu, N. L. Doh, and Z. Habib, "Optimal path planning in cluttered environment using RRT-AB," *Intell. Service Robot.*, vol. 11, no. 1, pp. 41–52, Jan. 2018.
[17] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli, and S. Teller, "Anytime motion planning using the RRT," in *Proc. IEEE Int. Conf. Robot. Autom.*, China, May 2011, pp. 1478–1483.
[18] J. Fan, X. Chen, Y. Wang, and X. Chen, "UAV trajectory planning in cluttered environments based on PF-RRT algorithm with goal-biased strategy," *Eng. Appl. Artif. Intell.*, vol. 114, Sep. 2022, Art. no. 105182.
[19] B. Ichter and M. Pavone, "Robot motion planning in learned latent spaces," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2407–2414, Jul. 2019.
[20] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, "Motion planning networks: Bridging the gap between learning-based and classical motion planners," *IEEE Trans. Robot.*, vol. 37, no. 1, pp. 48–66, Feb. 2021.
[21] F. Kiani, A. Seyyedabbasi, R. Aliyev, M. U. Gulle, H. Basyildiz, and M. A. Shah, "Adapted-RRT: Novel hybrid method to solve three-dimensional path planning problem using sampling and Metaheuristic-based algorithms," *Neural Comput. Appl.*, vol. 33, no. 22, pp. 15569–15599, Nov. 2021.
[22] F. Meng, L. Chen, H. Ma, J. Wang, and M. Q.-H. Meng, "Learning-based risk-bounded path planning under environmental uncertainty," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 4460–4470, Jul. 2024.
[23] F. Meng, J. Liu, H. Shi, H. Ma, H. Ren, and M. Q.-H. Meng, "Online time-informed kinodynamic motion planning of nonlinear systems," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 9589–9596, Nov. 2024.
[24] N. Li and S. I. Han, "Adaptive bi-directional RRT algorithm for three-dimensional path planning of unmanned aerial vehicles in complex environments," *IEEE Access*, vol. 13, pp. 23748–23767, 2025.
[25] J. Fan, X. Chen, and X. Liang, "UAV trajectory planning based on bi-directional APF-RRT algorithm with goal-biased," *Expert Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 119137.
[26] B. Wang, D. Ju, F. Xu, C. Feng, and G. Xun, "CAF-RRT: A 2D path planning algorithm based on circular arc fillet method," *IEEE Access*, vol. 10, pp. 127168–127181, 2022.
[27] C. Urmson and R. Simmons, "Approaches for heuristically biasing RRT growth," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 2, Dec. 2003, pp. 1178–1183.
[28] L. E. Dubins, "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents," *Amer. J. Math.*, vol. 79, no. 3, pp. 497–516, Jul. 1957.
[29] I. Lugo-Cárdenas, G. Flores, S. Salazar, and R. Lozano, "Dubins path generation for a fixed wing UAV," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, May 2014, pp. 339–346.
[30] S. Park, "Three-dimensional dubins-path-guided continuous curvature path smoothing," *Appl. Sci.*, vol. 12, no. 22, p. 11336, Nov. 2022.
[31] Y. Yang, H. Leeghim, and D. Kim, "Dubins path-oriented rapidly exploring random tree for three-dimensional path planning of unmanned aerial vehicles," *Electronics*, vol. 11, no. 15, p. 2338, Jul. 2022.
[32] J. S. Dai, "Euler–Rodrigues formula variations, quaternion conjugation and intrinsic connections," *Mechanism Mach. Theory*, vol. 92, pp. 144–152, Oct. 2015.
[33] I.-B. Jeong, S.-J. Lee, and J.-H. Kim, "Quick-RRT: Triangular inequality-based implementation of RRT with improved initial solution and convergence rate," *Expert Syst. Appl.*, vol. 123, pp. 82–90, Jun. 2019.
[34] B. Liao, F. Wan, Y. Hua, R. Ma, S. Zhu, and X. Qing, "F-RRT: An improved path planning algorithm with improved initial solution and convergence rate," *Expert Syst. Appl.*, vol. 184, Dec. 2021, Art. no. 115457.
[35] C. Wong, E. Yang, X.-T. Yan, and D. Gu, "Optimal path planning based on a multi-tree T-RRT approach for robotic task planning in continuous cost spaces," in *Proc. 12th France-Jpn. 10th Europe-Asia Congr. Mechatronics*, Sep. 2018, pp. 242–247.
[36] K. Yang and S. Sukkarieh, "An analytical continuous-curvature path-smoothing algorithm," *IEEE Trans. Robot.*, vol. 26, no. 3, pp. 561–568, Jun. 2010.

YEKYUNG JUNG received the bachelor's degree in mechanical engineering from Seoul National University of Science and Technology, South Korea, in 2023, where she is currently pursuing the master's degree with the Department of Defence AI Convergence Engineering. Her research interest includes path planning of unmanned aerial vehicles.

BEOM-SEOK OH (Member, IEEE) received the B.S. degree in computer science from Konkuk University, South Korea, in 2008, and the M.S. degree in biometrics and the Ph.D. degree in electrical and electronic engineering from Yonsei University, South Korea, in February 2010 and August 2015, respectively. From April 2015 to August 2019, he was a Research Fellow with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. Subsequently, he joined Tongmyong University, Busan, South Korea, and later moved to the Department of Computer Science, Gyeongsang National University, Jinju, South Korea. Since August 2022, he has been with the Department of Applied Artificial Intelligence, Seoul National University of Science and Technology, Seoul, South Korea, where he is currently an Associate Professor. His research interests include pattern analysis and classification, and machine learning.